



JEE Math Tutor

Architecture Deep Dive

Multi-Agent LangGraph Pipeline
Hybrid CRAG · Tavily MCP · Long-Term Memory

TECH STACK

LangGraph

LLaMA 3.3 70B

Redis Stack

FAISS + BM25

Cohere Embed

Tavily MCP

Streamlit

SymPy

SYSTEM OVERVIEW

Architecture



Streamlit Frontend

Chat UI · Activity panel
PDF upload · OAuth login
HITL forms · Memory viz



LangGraph StateGraph

14 nodes · ReAct loop
RedisSaver checkpoints
Interrupt() for HITL



Redis Stack

STM checkpoints · LTM
Vector HNSW index
User & thread registry

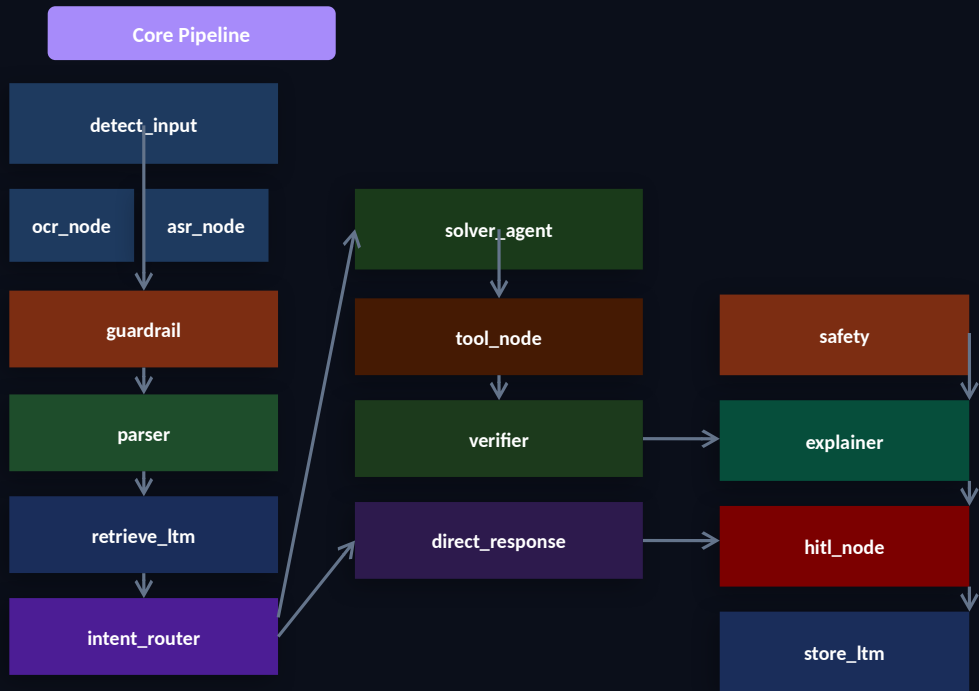


Tool Layer

Hybrid CRAG (RAG)
Tavily MCP web search
SymPy calculator

Student → Frontend → LangGraph → Redis + Tools → Response → Student

AGENT PIPELINE — 14 NODE LANGGRAPH



N O D E L E G E N D

 **Input Nodes**
detect, OCR, ASR

 **Guard & Parse**
guardrail, parser

 **Memory**
retrieve_ltm, store_ltm

 **Solver Pipeline**
solver ↔ tools ↔ verifier

 **Direct Response**
explain/hint/research

 **Safety + Explain**
output check + tutor

 **HITL**
interrupt() checkpoint

HYBRID CRAG — CORRECTIVE RAG PIPELINE

RAG System

1



PDF Ingestion

PyPDFLoader → chunk 800
chars
150 overlap · per thread_id
Append-safe (multi-PDF)

2



Dense Retrieval

Cohere embed-english-v3.0
1024-dim L2-normalised
FAISS IndexFlatIP · top 10

3



Sparse Retrieval

BM25Okapi on tokenized
chunks
Top 10 by BM25 score
Keyword complement to
dense

4



RRF Fusion

Reciprocal Rank Fusion
score = $1/(K+60 + \text{rank})$
Merge dense + sparse ranks

5



Corrective Filter

Drop cosine < 0.30
Off-topic chunks removed
Returns top-5 passages

Two-Call Pattern:

Call 1 forces rag_tool via tool_choice → RAG executes inline → result injected as HumanMessage sentinel. Call 2 binds only [calc, web_search] — writes full solution. Avoids Groq's ToolMessage validation error.

TAVILY MCP — REAL-TIME WEB SEARCH

Web Search

Request Flow



Solver Agent

Decides web search needed



tool_choice: web_search_tool

LangChain tool binding



FastMCP Client

Async SSE to mcp.tavily.com



Tavily MCP Server

search_depth=advanced, 5 results



Formatted Markdown

Direct answer + ranked snippets

Multi-Query Strategy

Query 1

Core formula / theorem / concept

Query 2

Worked example or step-by-step solution

Query 3

Edge case / common mistake / application

WHEN TO CALL

✓ Recent JEE questions

✓ Olympiad problems

✓ CRAG returned empty

✓ Current discoveries

WHEN NOT TO CALL

✗ Basic math computation

✗ Topics in uploaded PDF

✗ Standard integrals

✗ Routine algebra

AGENT ACTIVITY PANEL — LIVE STREAMING UI

Frontend

Live Activity Panel (3:7 column layout)

 detect_input mode=text	✓
 guardrail_agent PASSED topic=calculus	✓
 parser_agent Topic: Calculus	✓
 retrieve_ltm 3 episodes weak: calculus	✓
 intent_router solve hard power rule	✓
 solver_agent Calling rag_tool...	●
 rag_tool query: integration by parts	◆

How the Activity Panel Works

st.empty() placeholder

Panel rewrites entire block on every chunk — no duplicate cards

stream_mode='updates'

chatbot.stream() yields per-node dicts caught by _handle_chunk()

agent_payload_log

Every node writes to state — fields shown in collapsible payload table

Tool card injection

direct_response_tool_calls signals web search without ToolNode

Status states

active (pulsing blue) → tool (orange) → done (green) → hitl (red)

mark_all_done()

Called at stream end to flip all remaining active cards to done

MEMORY ARCHITECTURE — STM & LTM

Memory System

SHORT-TERM MEMORY (STM)

RedisSaver Checkpointer:	Auto-checkpoints AgentState after every node
Key pattern:	checkpoint:<thread_id>*
TTL:	2 hours (STM_SUMMARY_TTL)
HITL Resume:	interrupt() pauses → Command(resume=) restores exactly
Token Trimming:	> 8k tokens → LLM summarizes → replaces old messages
Rolling Summary:	stm:summary:<thread_id> — persists across restarts in TTL
Always keep last N:	KEEP_LAST_N=6 messages always preserved verbatim

LONG-TERM MEMORY (LTM)

Episodic Memory



episodic:<sid>:<eid>



One record per solved problem
Cohere embedding (1024-dim)
HNSW vector search · 90d TTL
Decay: $e^{-(\text{days}/30)} \times \log(\text{access}+1)$

Semantic Memory



semantic:<student_id>



weak_topics {topic: fail_count}
strong_topics {topic: success_count}
mistake_patterns [{pattern, count}]
Permanent · no TTL

Procedural Memory



procedural:<student_id>

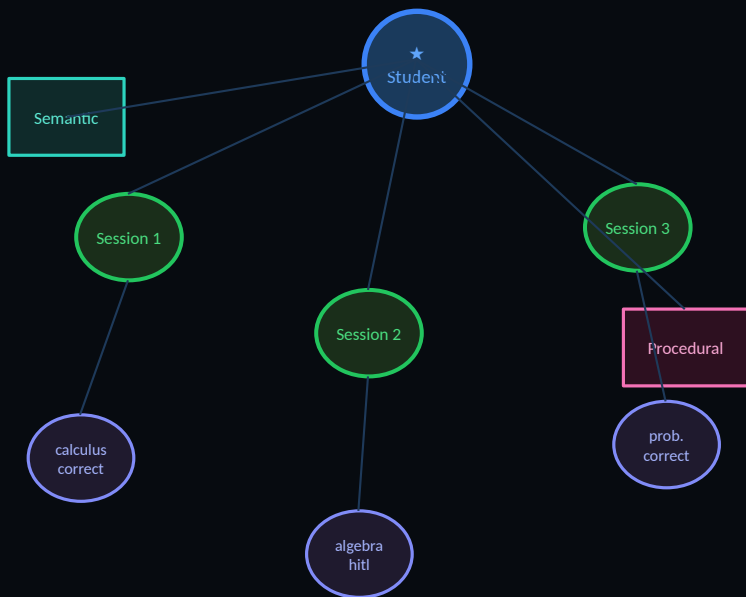
strategy_success per topic
success_rate + attempts_avg
Best = max(rate, -attempts)
Solver-flow only (flow gate)

⚠ Flow gate: semantic + procedural written ONLY for solve/hint/formula_lookup intents

MEMORY GRAPH VISUALIZER — NEO4J STYLE

Memory Viz

vis.js Interactive Graph



Visualizer Features

7 Node Types

student · session · agent · tool
episodic · semantic · procedural

Interactive

Click → detail panel
Double-click → expand neighbours
Hover → glow effect

Controls

4 layout presets (Radial, Hierarchical, Organic, Tight) · Physics toggle

Filters

Toggle node types · Max threads slider
Edge label toggle · PNG export

Decay Color

Green ≥ 0.6 · Yellow ≥ 0.3 · Red < 0.3
Spaced-repetition forgetting curve

INTENT ROUTING & DUAL PIPELINE

Routing

6 INTENT TYPES



solve

Full ReAct pipeline



explain

Direct response



hint

Solver pipeline



formula_lookup

Direct response



research

Web search + direct



generate

Web search + direct

PIPELINE ROUTING

Solve Path (solve · hint · formula_lookup)

intent_router → solver_agent ↔ tool_node → verifier_agent → safety_agent →
explainer_agent → hitl_node → store_ltm

Up to 3 retry iterations · Verifier routes correct/incorrect/needs_human

Direct Response Path

explain · research · generate

1

Intent identified

router → direct_response_node

2

Web search (optional)

research/generate → Tavily MCP before LLM call

3

LLM call

reserve_llm + XML-tagged <content> response

4

Stub outputs

solver_output + verifier_output stubs so store_ltm won't crash

5

Safety check

Same safety_agent as solve path

6

HITL satisfaction

hitl_node(satisfaction) → student confirms or asks follow-up

7

Store LTM

Episodic only — semantic/procedural skipped (flow gate)

REDIS STORAGE SCHEMA & SYSTEM SUMMARY

Redis Stack

REDIS KEY PATTERNS

checkpoint:<thread_id>:*

LangGraph STM state snapshots

stm:summary:<thread_id>

Rolling LLM summary · TTL 2h

user:<student_id>

Hash: name, email, problems_solved

threads:<student_id>

Sorted set scored by timestamp

thread:<thread_id>:meta

Hash: summary, topic, outcome

episodic:<sid>:<eid>

JSON + HNSW vector · TTL 90d

idx:episodic

RedisVL HNSW index (COSINE/FLOAT32)

semantic:<student_id>

JSON: weak/strong/mistakes · no TTL

procedural:<student_id>

JSON: strategy_success · no TTL

SYSTEM AT A GLANCE

14

LangGraph nodes

6

Intent types

3

Memory layers

2

Groq API keys

1024

Embedding dims

0.30

CRAG min cosine

KNOWN LIMITATIONS

⚠ In-memory FAISS index lost on restart — re-upload PDF

⚠ Groq rate limits on free tier — two-key architecture mitigates

⚠ `weak_topics` needs retry sessions to populate

⚠ No concurrent multi-user scaling in single Streamlit process